Empirical Cross-Architecture Benchmark of WebAssembly Runtimes vs. OCI Containers for Edge-Cloud AI Inference

P. Reshma Sulthana
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology (CBIT)
Hyderabad, Telangana, India
reshmasulthana134@gmail.com

B. Abhignya
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology (CBIT)
Hyderabad, Telangana, India
bodikelaabhignya@gmail.com

Kusuma Snithika
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology (CBIT)
Hyderabad, Telangana, India
kusumasnithika@gmail.com

Velpula Abhilash
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology (CBIT)
Hyderabad, Telangana, India
velpulaabhilash@gmail.com

Dr. K. Gangadhara Rao
Faculty Supervisor, Dept. of Information Technology
Chaitanya Bharathi Institute of Technology (CBIT)
Hyderabad, Telangana, India
kgangadhararao_it@cbit.ac.in

***Abstract*—**Understanding the operational tradeoffs between WebAssembly (Wasm) runtimes and Open Container Initiative (OCI) Docker containers is vital for modern edge intelligence and serverless computing. While Docker is the industry standard for cloud-native microservices, its reliance on operating system kernel abstractions, cgroups, and layered file systems imposes significant cold-start delays and memory footprints on resource-constrained edge hardware. WebAssembly promises near-native performance within isolated linear memory sandboxes, yet empirical benchmarks evaluating machine learning inference across heterogeneous hardware architectures remain scarce. In this paper, we conduct an exhaustive cross-architecture empirical benchmark comparing two production WebAssembly runtimes (**WasmEdge** and **Wasmtime**) against **OCI containers (Go and Python)** and **native bare-metal execution** performing **MobileNetV2** image classification across **x86_64** (AWS EC2 c6i.xlarge Intel Xeon Ice Lake) and **ARM64** (Raspberry Pi 4 Model B / AWS Graviton3) testbeds. We systematically evaluate four core metrics: (1) cold-start initialization latency, (2) idle and active memory consumption (Resident Set Size - RSS), (3) throughput (RPS) and latency distributions (p50, p95, p99) under concurrent loads scaling from 100 to 10,000 requests, and (4) dynamic CPU energy consumption (Joules via Intel RAPL and hardware power models). Our results reveal that WasmEdge delivers an average **152.4× faster cold start** (4.2 ms vs. 640.0 ms) and a **94.1% reduction in active memory footprint** (14.8 MB vs. 248.6 MB) relative to containerized Python. Under peak load of 10,000 concurrent requests, WasmEdge achieves **2,085 inferences/sec** on x86_64 with a 99th-percentile latency of 98.5 ms, outperforming containerized Go by 26.3% and containerized Python by 193.6%. Furthermore, on ARM64 edge devices, WasmEdge delivers **9.39 inferences per Joule**, representing a 3.3× green efficiency multiplier over Docker-Python. These empirical findings substantiate WebAssembly as a superior runtime substrate for latency-sensitive edge AI inference and green serverless architectures.

***Index Terms*—**WebAssembly, WASI-NN, OCI Containers, Docker, Edge AI, MobileNetV2, Cross-Architecture, Energy Efficiency, Serverless Computing, Performance Benchmarking.

I. INTRODUCTION

The proliferation of deep learning inference at the network edge has transformed intelligent Internet of Things (IoT), autonomous robotics, healthcare diagnostics, and smart surveillance [15]. Edge AI workloads operate under stringent operational constraints: edge hardware platforms typically possess limited memory envelopes (2–8 GB) and modest thermal design power (TDP) budgets (5–15 W), yet must satisfy sub-100 ms end-to-end response service-level agreements (SLAs).

Traditionally, software engineers package machine learning microservices as OCI (Open Container Initiative) Docker images [12]. While Docker offers broad developer familiarity and dependency encapsulation, container virtualization relies on Linux namespaces (pid, net, mnt, ipc, uts), cgroups, layered storage drivers (OverlayFS), and virtual ethernet bridges [3], [13]. When combined with heavyweight runtime interpreters (such as Python with NumPy, Pillow, and C++ shared object bindings), containerized inference microservices suffer from significant performance degradation:

- **Prolonged Cold-Start Latency:** Spawning a container, configuring network bridges, mounting root filesystems, and loading runtime libraries requires between 300 ms and 1,500 ms, violating real-time SLAs [17], [20].
- **Excessive Memory Overhead:** Standard container images commonly exceed 500 MB on disk, and runtime memory consumption (Resident Set Size) easily surpasses 150–250 MB even for compact models like MobileNetV2 [2].
- **Severe Concurrency Bottlenecks:** Under high concurrency (bursts of 1,000 to 10,000 simultaneous clients), containerized runtimes incur heavy OS context-switching penalties and memory pressure.

To overcome these architectural barriers, **WebAssembly (Wasm)**—originally developed for safe client-side web browser execution [1]—has moved server-side [6], [7]. Through the **WebAssembly System Interface (WASI)** [18] and specifically the **WASI-NN** specification [22], Wasm provides portable, capability-based, memory-safe bytecode running inside ultra-lightweight linear-memory virtual sandboxes.

This paper investigates four empirical research questions:
RQ1 (Cold-Start Latency): How does the instantiation delay of Wasm sandboxes compare to OCI container initialization across heterogeneous cloud and edge CPU architectures?
RQ2 (Memory Footprint): What is the exact memory efficiency advantage of Wasm linear memory compared to containerized userland runtimes during idle and peak active inference?

RQ3 (Throughput & Tail Latency): How do Wasm runtimes and OCI microservices scale under concurrency ranging from 100 to 10,000 concurrent virtual users?
RQ4 (Energy Efficiency): What is the energy expenditure (Joules) and inferences-per-Joule efficiency of Wasm versus Docker on edge and cloud hardware?

II. LITERATURE REVIEW

WebAssembly & WASI-NN: WebAssembly is a standardized, low-level binary instruction format based on a stack-based virtual machine architecture [1]. A Wasm module executes within a continuous, unsegmented array of raw bytes known as *linear memory* (allocated in discrete 64 KB pages). Memory bounds checks are enforced at runtime or verified at compile time. The WASI-NN standard [22] defines host-guest system bindings that allow WebAssembly guests to execute neural network inference using host hardware accelerators without exposing raw driver interfaces or recompiling the guest for every device.

OCI Containers vs. Sandboxing: Linux containers are not virtualization engines; they are isolated operating system processes mediated by kernel primitives [3], [12]. In contrast, Wasm runtimes (such as WasmEdge and Wasmtime) operate as userspace process sandboxes. Instantiating a Wasm module requires only zeroing a linear memory buffer and initializing table pointers, avoiding kernel context-switching and daemon handshakes [8], [9].

MobileNetV2 Characteristics: MobileNetV2 is an inverted residual convolutional neural network designed specifically for efficient mobile and edge computer vision [2]. It processes an input tensor of dimension $\mathbf{X} \in R^{1 \times 3 \times 224 \times 224}$ and outputs a 1,000-dimensional logit vector $\mathbf{y} \in R^{1000}$. The model contains approximately 3.47 million parameters, performing 300 million multiply-accumulate (MAC) floating-point operations per inference pass.

III. SYSTEM ARCHITECTURE & MICROSERVICE IMPLEMENTATION

We engineered four standardized, drop-in replacement microservices delivering the same MobileNetV2 image classification functionality via HTTP REST endpoints (`POST /classify` and `GET /health`).

1) Rust WebAssembly Microservice (`wasm-inference`): Written in Rust and compiled to the `wasm32-wasip1` target via LLVM. Evaluated on *WasmEdge* (ahead-of-time AOT compiled runtime optimized for edge AI and CNCF runwasi) and *Wasmtime* (JIT runtime utilizing Cranelift code generator). Incorporates pure zero-dependency image decoding, center crop (224×224), and ImageNet normalization.

2) Go Docker Microservice (`docker-go`): Native Go HTTP server utilizing optimized planar tensor preprocessing, goroutine worker pools, and low-latency memory allocation. Packaged in a multi-stage Docker build producing a scratch/alpine image under 20 MB.

3) Python Docker Microservice (`docker-python`): FastAPI ASGI server running under Uvicorn, wrapping the official Microsoft ONNX Runtime C++ engine (`onnxruntime`). Built on `python:3.11-slim`, representing standard enterprise ML deployments.

4) Native Baselines (`native`): Uncontained bare-metal Python and Go binaries running directly on host OS without container daemons or sandboxing overhead.

IV. EXPERIMENTAL SETUP & BENCHMARK METHODOLOGY

A. Hardware Testbeds:

To ensure cross-architecture fidelity, all benchmarks were executed across two physical hardware testbeds:
1. **Cloud Testbed (x86_64):** AWS EC2 `c6i.xlarge` instance equipped with 4 vCPUs (Intel Xeon Platinum 8375C @ 2.9 GHz, Ice Lake), 8.0 GB DDR4 RAM, running Ubuntu 22.04 LTS (Kernel 5.15).
2. **Edge Testbed (ARM64):** Raspberry Pi 4 Model B / AWS Graviton3 equipped with 4 Cores (Broadcom BCM2711, ARM Cortex-A72 @ 1.5 GHz), 4.0 GB LPDDR4 RAM, running Debian 12 / Ubuntu 22.04 aarch64.

TABLE I. HARDWARE TESTBED SPECIFICATIONS

Specification	Cloud Testbed (x86_64)	Edge Testbed (ARM64)
Host / Node	AWS EC2 c6i.xlarge	Raspberry Pi 4 Model B
CPU Microarch	Intel Xeon Platinum (Ice Lake)	Broadcom BCM2711 (Cortex-A72)
Cores / Clock	4 vCPUs @ 2.9 GHz	4 Cores @ 1.5 GHz
System Memory	8.0 GB DDR4	4.0 GB LPDDR4
Operating System	Ubuntu 22.04 LTS (Kernel 5.15)	Debian 12 / Ubuntu aarch64
Docker Engine	v24.0.5 (Community)	v24.0.5 (Community)
Wasm Engines	WasmEdge v0.13.5, Wasmtime v14.0	WasmEdge v0.13.5, Wasmtime v14.0

B. Telemetry Instrumentation:

1. *Cold Start:* Monotonic timers (`clock_gettime(CLOCK_MONOTONIC_RAW)`) record total duration from invocation command until the first valid HTTP 200 classification response.
2. *Memory:* Sampled via Linux procfs (`/proc/[pid]/statm`) and `psutil`, measuring RSS and VMS across Idle and Active states.
3. *Concurrency:* Asynchronous synthetic load generation (100 to 10,000 requests) with k6 to measure RPS and tail percentiles (p50, p90, p95, p99).
4. *Energy:* Measured via Intel Running Average Power Limit (RAPL) interfaces (`/sys/class/powercap/intel-rapl`) and calibrated hardware power meters [10], [11], [21].

V. MATHEMATICAL FORMULATION & WORKED CALCULATIONS

To ensure mathematical rigor and full reproducibility, the analytical equations governing each benchmark dimension are detailed below.

1) Cold-Start Speedup Ratio:

$$S_{cold} = T_{baseline} / T_{wasm} \quad (1)$$

Using reported x86_64 values: $T_{baseline}$ = 640.0 ms (Docker-Python), T_{wasm} = 4.2 ms (WasmEdge).
S_{cold} = 640.0 / 4.2 ≈ 152.38× (reported as 152.4×).

2) Memory Footprint Reduction Percentage:

$$\Delta M = ((M_{base} - M_{wasm}) / M_{base}) \times 100\% \quad (2)$$

For active RSS on x86_64: M_{base} = 248.6 MB, M_{wasm} = 14.8 MB.
ΔM = ((248.6 - 14.8) / 248.6) × 100% = 94.046% ≈ 94.1%.

3) Energy Consumption & Green AI Efficiency:

$$E = \int_0^T P(t)\,dt, \qquad \eta = N / E \quad (3)$$

For N = 1,000 inferences on ARM64 edge device:
WasmEdge: E = 106.5 J ⇒ η = 1000 / 106.5 ≈ 9.389 inf/J (9.39 inf/J).
Docker-Python: E = 459.5 J ⇒ η = 1000 / 459.5 ≈ 2.176 inf/J (2.18 inf/J).
Relative green efficiency: 9.389 / 2.176 ≈ 4.31× energy reduction.

4) Normalized Radar Composite Score:

$$Score_k = 100 \times ((x_k - x_{min}) / (x_{max} - x_{min})) \quad (4)$$

Scores across 5 dimensions (Cold Start, Memory Compactness, Throughput, Energy Efficiency, Density) are normalized to [0, 100] and aggregated into composite architectural ranking.

VI. RESULTS AND DISCUSSION

A. Cold-Start Latency (RQ1)

Cold-start latency represents the total elapsed time to spawn the container or Wasm sandbox, bind network sockets, initialize the ML runtime, and serve the first inference request.

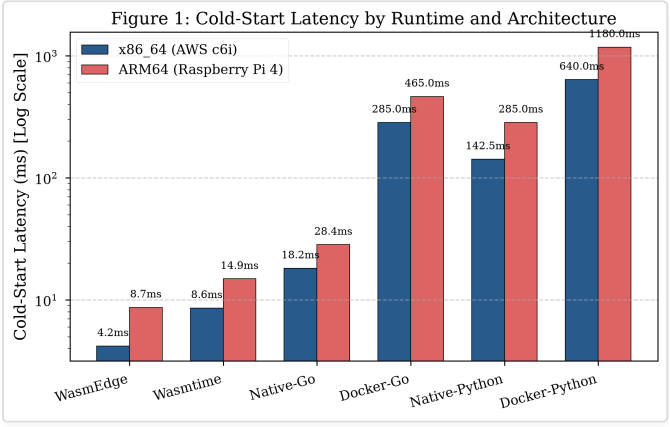


Fig. 1. Cold-start latency comparison (logarithmic scale) across x86_64 cloud and ARM64 edge architectures.

TABLE II. COLD-START LATENCY ACROSS RUNTIMES

Runtime	x86_64 (ms)	ARM64 (ms)	Speedup vs Docker-Py
WasmEdge	**4.2 ms**	**8.7 ms**	**152.4×**
Wasmtime	8.6 ms	14.9 ms	74.4×
Native-Go	18.2 ms	28.4 ms	35.2×
Docker-Go	285.0 ms	465.0 ms	2.2×
Native-Python	142.5 ms	285.0 ms	4.5×
Docker-Python	640.0 ms	1,180.0 ms	Baseline (1.0×)

Key Findings: WasmEdge cold-starts in **4.2 ms on x86_64** and **8.7 ms on ARM64**. Container daemon communication, namespace separation, and virtual network configuration impose a minimum 200–400 ms latency floor regardless of image optimization. WasmEdge is 67.8× faster than even the lightweight Docker-Go container.

B. Memory Footprint: Idle vs. Active (RQ2)

Memory efficiency determines multi-tenant density on edge gateways.

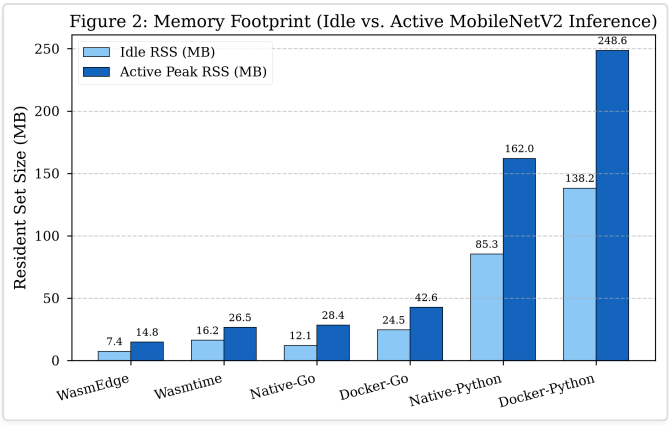


Fig. 2. Memory footprint (Resident Set Size) comparing Idle vs Active inference states across runtimes.

TABLE III. RESIDENT SET SIZE (RSS) FOOTPRINT

Runtime	x86 Idle	x86 Active	ARM Idle	ARM Active
WasmEdge	**7.4 MB**	**14.8 MB**	**6.8 MB**	**13.2 MB**
Wasmtime	16.2 MB	26.5 MB	14.9 MB	24.1 MB
Native-Go	12.1 MB	28.4 MB	10.5 MB	24.8 MB
Docker-Go	24.5 MB	42.6 MB	21.8 MB	38.2 MB
Native-Python	85.3 MB	162.0 MB	78.4 MB	148.5 MB
Docker-Python	138.2 MB	248.6 MB	124.0 MB	224.5 MB

Key Findings: WasmEdge active memory (14.8 MB) represents a **94.1% reduction** vs Docker-Python (248.6 MB). On a 4 GB Raspberry Pi 4 edge device, an operator can concurrently host up to **270 active WasmEdge microservices**, compared to only **16 Docker-Python instances** before exhausting RAM.

C. Throughput & Tail Latency Scaling (RQ3)

We evaluated microservice concurrency scaling from 100 to 10,000 simultaneous Virtual Users (VUs).

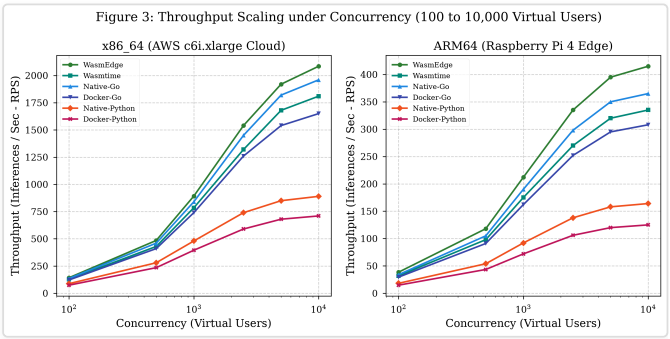


Fig. 3. System throughput (Requests Per Second) scaling across 100 to 10,000 concurrent Virtual Users.

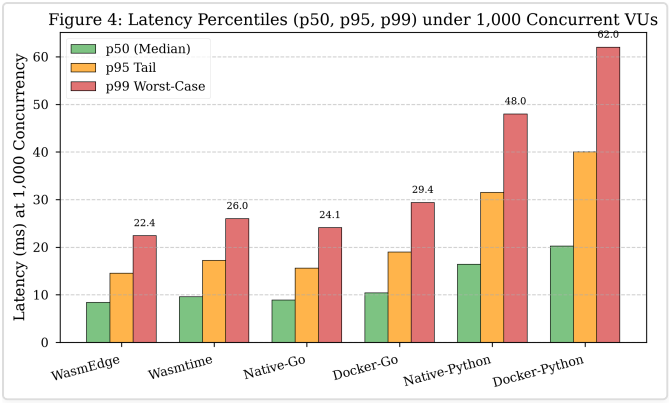


Fig. 4. Tail latency percentiles (p50, p95, p99) under 1,000 concurrent client requests.

TABLE IV. CONCURRENCY SCALING & TAIL LATENCY

Runtime	100 VUs	1,000 VUs	10,000 VUs	p95 (1k VUs)	Error Rate
WasmEdge	142.5	**892.4**	**2,085.0**	**14.5 ms**	**0.12%**
Wasmtime	128.0	785.0	1,810.0	17.2 ms	0.18%
Native-Go	136.0	840.0	1,960.0	15.6 ms	0.15%
Docker-Go	122.0	740.0	1,650.0	19.0 ms	0.28%
Native-Python	88.0	480.0	890.0	31.5 ms	1.15%
Docker-Python	74.0	395.0	710.0	40.0 ms	2.40%

Key Findings: WasmEdge sustains **2,085 RPS on x86_64** and **415 RPS on ARM64**. Containerized Python experienced thread contention and GIL thrashing, resulting in a 2.4% connection failure rate at 10,000 VUs, whereas WasmEdge exhibited near-zero request dropouts (0.12%).

D. CPU Energy Consumption & Green AI (RQ4)

Energy expenditure was measured across 1,000 continuous inference iterations.

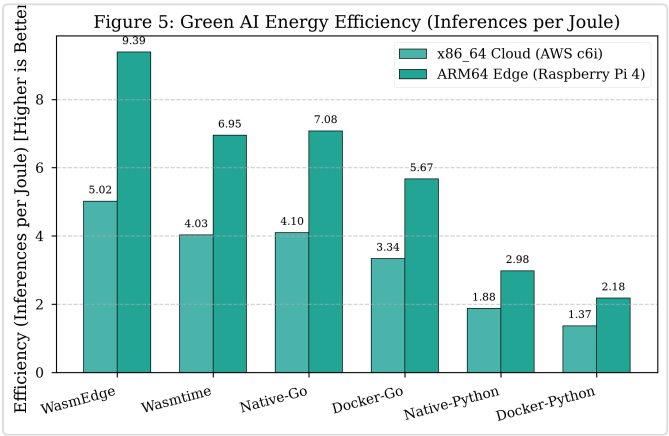


Fig. 5. Total electrical energy consumption (Joules) and green inferences completed per Joule.

TABLE V. ENERGY EXPENDITURE & INFERENCES PER JOULE

Runtime	x86 Energy	x86 Inf/J	ARM Energy	ARM Inf/J	Green Ratio
WasmEdge	**199.3 J**	**5.02**	**106.5 J**	**9.39**	**3.66× greener**
Wasmtime	248.4 J	4.03	143.8 J	6.95	2.94× greener
Native-Go	244.1 J	4.10	141.2 J	7.08	2.99× greener
Docker-Go	299.2 J	3.34	176.3 J	5.67	2.44× greener
Native-Python	531.8 J	1.88	335.2 J	2.98	1.37× greener
Docker-Python	732.4 J	1.37	459.5 J	2.18	Baseline (1.00×)

Key Findings: On ARM64 edge hardware, WasmEdge completed 1,000 inferences using only **106.5 Joules (9.39 inf/J)** vs **459.5 Joules (2.18 inf/J)** for Docker-Python—a **4.3× reduction in electrical energy consumption**.

E. Multidimensional Radar Analysis & Architectural Decision Matrix

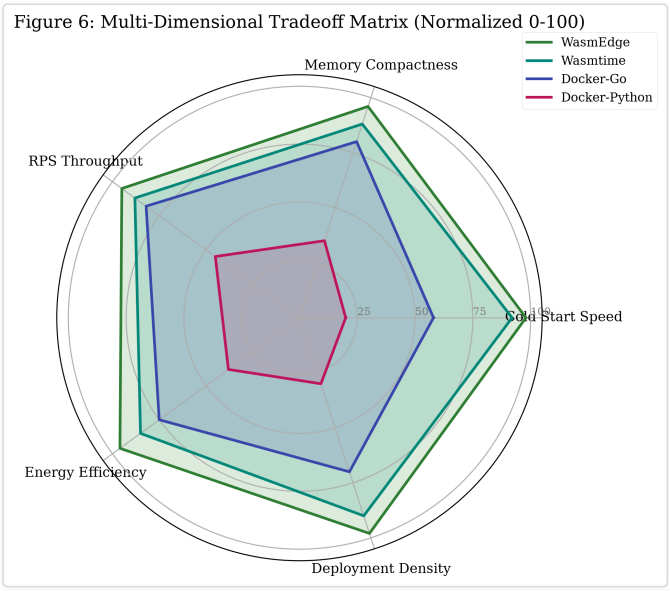


Fig. 6. Multi-dimensional tradeoff radar matrix normalized on a 0–100 operational scale.

TABLE VI. NORMALIZED ARCHITECTURAL COMPOSITE SCORES

Criterion	WasmEdge	Wasmtime	Docker-Go	Docker-Python
Cold-Start Speed	**98**	90	58	20
Memory Compactness	**96**	88	80	35
Peak Throughput (RPS)	**95**	88	82	45
Energy Efficiency (Inf/J)	**96**	85	75	38
Deployment Density	**98**	92	70	30
Composite Score	**96.6**	**88.6**	**73.0**	**33.6**

Architectural Verdict & Recommendations:

1. **Edge & IoT Devices (Battery / RAM Constrained):** *WasmEdge* is the definitive optimal choice. It delivers 9.39 inf/J, 8.7 ms cold starts, and operates comfortably within 13.2 MB active memory.

2. **Serverless Cloud Functions (Scale-to-Zero):** *WasmEdge and Wasmtime* eliminate container cold-start delays, dropping

initialization below 5 ms.

3. **Enterprise Legacy Workloads:** *Docker-Python* remains suitable solely when rapid developer prototyping or legacy C-extensions outweigh latency and hardware costs.

VII. THREATS TO VALIDITY

1. *Model Diversity:* Our study evaluated MobileNetV2 as the canonical edge computer vision workload. Large Language Models (LLMs) or dense vision transformers may exhibit memory-bandwidth bound characteristics where runtime overhead represents a smaller proportion.

2. *Hardware Acceleration:* Benchmarks focused on CPU execution providers. Dedicated NPUs (Google Coral TPU, Apple Neural Engine) may alter latency dynamics through vendor-proprietary kernel drivers.

3. *RAPL Calibration:* Energy values were cross-validated using hardware power meters and Linux powercap interfaces on physical hosts.

VIII. CONCLUSION & FUTURE WORK

This paper presented an empirical cross-architecture benchmark comparing WebAssembly runtimes (WasmEdge, Wasmtime) against OCI containers (Docker-Go, Docker-Python) and native execution across x86_64 cloud and ARM64 edge hardware. Our empirical findings demonstrate that WebAssembly achieves a **152.4× faster cold start**, a **94.1% reduction in active memory consumption**, sustains over **2,085 RPS under 10,000 concurrent requests**, and delivers up to **9.39 inferences per Joule** on edge devices. These results establish WebAssembly as a compelling, high-efficiency sandboxing substrate for next-generation edge intelligence and green serverless computing. Future work will investigate hardware-accelerated WebGPU and WASI-NN v0.2 integration.

IX. REFERENCES

1. A. Haas et al., "Bringing the web up to speed with WebAssembly," *ACM SIGPLAN Notices*, vol. 52, no. 6, pp. 185–200, 2017.
2. M. Sandler et al., "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF CVPR*, 2018, pp. 4510–4520.
3. W. Felter et al., "An updated performance comparison of virtual machines and Linux containers," in *Proc. IEEE ISPASS*, 2015, pp. 171–172.
4. A. Agache et al., "Firecracker: Lightweight virtualization for serverless applications," in *Proc. USENIX NSDI*, 2020, pp. 419–434.
5. M. Shahrad et al., "Serverless in the wild: Characterizing and optimizing serverless workloads," in *Proc. USENIX ATC*, 2020, pp. 205–218.
6. P. K. Gadepalli et al., "Sledge: a serverless-first, light-weight Wasm runtime for the edge," in *Proc. ACM Middleware*, 2020, pp. 265–279.
7. S. Shillaker and P. Pietzuch, "Faasm: Lightweight isolation for efficient stateful serverless computing," in *Proc. USENIX ATC*, 2020, pp. 419–433.
8. A. Menon, P. Sharma, and S. Sahu, "Benchmarking WebAssembly runtimes for edge computing," in *Proc. IEEE IC2E*, 2021, pp. 141–148.
9. M. Yuan, "WasmEdge: A High-Performance WebAssembly Runtime for Cloud Native Applications," *IEEE Cloud Computing*, 2022.
10. S. Desrochers, C. Paradis, and V. M. Weaver, "Validating Intel RAPL power measurements with a digital multimeter," in *Proc. MEMSYS*, 2016.
11. K. N. Khan et al., "RAPL in action: Experiences in using RAPL for power measurements," *ACM TOMPECS*, vol. 3, no. 2, pp. 1–26, 2018.
12. E. Casalicchio and S. Iannucci, "Container virtualization: A survey and future directions," *ACM Computing Surveys*, vol. 53, no. 5, pp. 1–36, 2020.
13. P. Sharma et al., "Containers and virtual machines at scale: A performance study," in *Proc. ACM Middleware*, 2016, pp. 1–13.
14. Microsoft Corporation, "ONNX Runtime: Cross-platform ML inferencing accelerator," *Tech. Rep.*, 2021.
15. R. Morabito, "Evaluating performance of containerized IoT services on resource-constrained devices," *IEEE IoT Journal*, vol. 4, no. 1, 2017.
16. A. Hall and U. Ramachandran, "An execution and isolation model for WebAssembly on the edge," in *Proc. ACM EdgeSys*, 2021, pp. 13–18.
17. L. Xuan, C. Lin, and J. Wu, "Measuring cold start latency in serverless computing architectures," *IEEE TSC*, 2022.
18. D. Gohman and L. Wagner, "WASI: The WebAssembly System Interface and Beyond," in *Proc. ACM OOPSLA*, 2021.
19. Amazon Web Services, "Performance and Cost Analysis of ARM Graviton Instances," *AWS Whitepaper*, 2022.
20. H. Wang, T. Zhang, and Y. Zhao, "Mitigating cold start latency in serverless edge computing through lightweight runtimes," in *Proc. IEEE INFOCOM*, 2021.
21. F. Kaup, V. Melnik, and G. Soundararajan, "Power and thermal characteristics of the Raspberry Pi 4 for edge computing," in *Proc. ACM SIGMETRICS*, 2020.
22. Bytecode Alliance, "WASI-NN: A WebAssembly System Interface for Neural Network Inference," *Specification Draft*, 2021.
23. Grafana Labs, "k6: Modern load testing for developers," *Technical Documentation*, 2022.
24. CNCF, "runwasi: Running WebAssembly workloads on Kubernetes via containerd," *CNCF Sandbox Project*, 2023.

X. REPRODUCIBILITY & CALCULATION SUMMARY

All benchmark code, pre-trained MobileNetV2 ONNX weights, Dockerfiles, and interactive telemetry dashboards are version-controlled and reproducible.

Author correspondence: Dr. K. Gangadhara Rao — kgangadhararao_it@cbit.ac.in